Continuum Architecture Interview Guide: Chosen vs Not Chosen Design Decisions

Layered AI Memory Architecture, Retrieval Strategy, Trade-offs, and Interview Defense

REPOSITORY
/Users/mayanksahu/Continuum

GENERATED
2026-05-25, Asia/Kolkata

EVIDENCE STANDARD
Implementation files, tests, configs, migrations, docs, benchmarks, and generated findings were inspected before writing.

USE CASE
SDE-2 / AI infrastructure / system design interview preparation.

This guide is deliberately not marketing copy. It separates implemented code from partial, planned, recommended, and not-chosen architecture.

Table of Contents

Current-State Disclaimer

Continuum is an evolving memory framework. The repository contains a modern packaged runtime under `continuum/`, a legacy STM/MTM agent stack under `memory/` and `agent/`, evaluation experiments under `evals/longmemeval/`, and benchmark artifacts under `bench/` and `results/`. This document treats those separately.

Implemented
Code exists in the runtime or tests exercise it directly.

Partially implemented
Schema, interface, test simulation, or limited path exists, but the full production behavior is not complete.

Planned
Docs, config, comments, or roadmap mention it; implementation is not present or not wired.

Recommended
A defensible next step based on the architecture and failure data.

Not chosen
An alternative rejected for this project or unsuitable for v1.

CURRENT REPO STATUS

The default `ContinuumSession` works with in-memory STM and a stub responder. MTM, LTM, retrieval, promotion, optimizer, policy engine, Postgres storage, and real responder are injected components. That is interface-first and useful, but it means the complete memory stack is not automatic by default.

CHAPTER 1 - Executive Summary

Continuum is a memory-centric AI agent framework. Its architectural center is not "ask a vector DB and stuff chunks into a prompt." The repository shows a layered memory design: raw short-term turns, medium-term summaries, long-term facts, hybrid retrieval, promotion decisions, policy-aware handling, context optimization, and an evaluation harness focused on failure classification.

Normal RAG is insufficient for long-running agents because it treats memory mostly as retrievable text. Agents need lifecycle: some facts should stay in STM only, some should be summarized into MTM, and only stable, source-backed, high-value facts should become LTM. Without lifecycle, permanent memory is polluted by transient chatter, outdated statements, duplicates, and contradictions.

The strongest current claim from the repo is: Continuum has a modular memory substrate with implemented STM/MTM/LTM stores, hybrid retrieval, promotion pipeline, context optimizer, policy engine, evaluation harness, and memory-operation benchmarks. The honest limitation is that several high-value intelligence layers are partial or planned: raw episode ledger wiring, robust supersession/update engine, dedicated temporal engine, preference resolver, core compiled wiki layer, production observability, and Continuum-Code scanner.

Area	Current State	Evidence from Repo	Interview Meaning
Session orchestration	Implemented	`continuum/core/session.py`	Async session wrapper; components are injected and optional.
STM	Implemented	`memory/stm/ConversationSTM.py`, `continuum/stores/stm/conversation_stm.py`, `PostgresSTM`	Session-local conversational memory is concrete.
MTM	Implemented	`continuum/stores/postgres/mtm.py`, legacy `memory/mtm/`	Medium-term summaries/blocks exist, with dedupe and processed markers.
LTM	Partial	`migrations/001_ltm_schema.sql`, `continuum/stores/postgres/ltm.py`	Durable fact store exists; full contradiction/supersession automation is not complete.
Hybrid retrieval	Implemented	`PostgresLTM.search_hybrid`, `PostgresRetriever.hybrid_search`	Dense + lexical RRF is real, with graph expansion and optional reranker.
Context optimizer	Implemented	`continuum/optimizer/`	Budget-aware context assembly is a product lever, not model compression.
LLM Wiki	Partial	`evals/longmemeval/wiki.py`, `bootstrap_ollama.py`	Implemented for eval experiments; not a core runtime source of truth.
Evaluation	Implemented	`evals/longmemeval/`, `bench/`, `findings/`	Can talk about failure-driven iteration with evidence.
Continuum-Code	Planned	`ContinuumCodeConfig`, `CodeMemoryPolicy`, docs state not implemented	Future extension, not current implementation.

90-SECOND INTERVIEW PITCH

Continuum is a memory infrastructure layer for AI agents. I designed it around memory lifecycle rather than treating everything as flat RAG chunks. The current runtime has STM for raw recent turns, MTM for summary-like intermediate memory, and PostgreSQL/pgvector LTM for durable facts with metadata, confidence, validity windows, invalidation, and graph edges. Retrieval is hybrid: dense vector search, lexical pg_trgm search, RRF fusion, composite scoring, optional graph expansion, and optional reranking. The architecture deliberately keeps raw memory and derived artifacts separate, because summaries, vectors, and wiki pages are lossy. The repo also includes LongMemEval and memory-operation benchmarks that show a critical lesson: high recall does not guarantee answer accuracy. The next hard work is temporal reasoning, knowledge-update handling, preference memory, and stronger validation, not just bigger embeddings.

CHAPTER 2 - Repository-Grounded Architecture Map

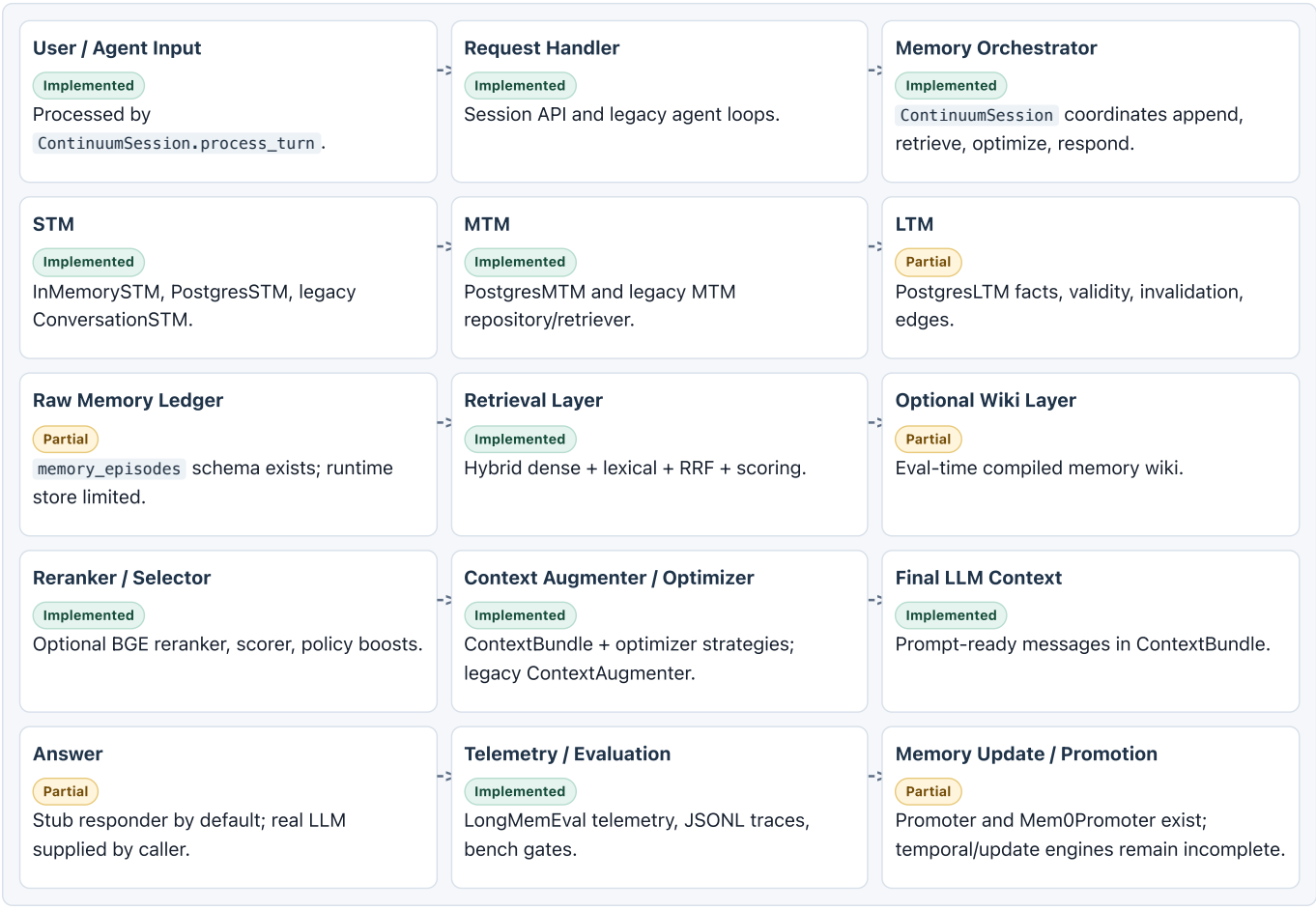
This map lists components proven by repository files. "Partial" means there is code or schema, but the implementation is limited, eval-only, not packaged, or not wired by default.

Component	File/Path	Responsibility	Status	Notes
ContinuumSession	`continuum/core/session.py`	Async session lifecycle, STM append, retrieval, responder call, background bookkeeping, checkpoint/search APIs.	Implemented	Defaults to in-memory STM and stub responder; other tiers are injected.
Core types	`continuum/core/types.py`	MemoryItem, Query, TokenBudget, ContextBundle, ScoreBreakdown, Promotion models.	Implemented	Defines STM/MTM/LTM contract vocabulary.
Protocols	`continuum/core/protocols.py`	STM/MTM/LTM/Retriever/Promoter/Optimizer interfaces.	Implemented	Interface-first architecture; runtime_checkable protocols used in tests.
BackgroundQueue	`continuum/core/background.py`	Bounded async queue, retries, dead-letter, stats/health.	Implemented	Used by session for non-blocking bookkeeping.
ConversationSTM	`memory/stm/ConversationSTM.py`	Bounded STM with token counting, eviction, compression hooks, rollback, stats.	Implemented	Legacy implementation re-used by modern wrapper.
InMemorySTM	`continuum/stores/stm/conversation_stm.py`	Async STMProtocol adapter over legacy ConversationSTM per session.	Implemented	Uses asyncio lock; supports flush_to(MTM).
ThreadSafeSTM	`memory/stm/ThreadSafeSTM.py`, `continuum/stores/stm/thread_safe_stm.py`	Sync/async thread-safety wrappers.	Implemented	Modern class is `AsyncSafeSTM`.
SharedSTM	`memory/stm/SharedSTM.py`	Filtered shared STM views for agents.	Partial	Legacy path; useful concept but not central packaged API.
PostgresSTM	`continuum/stores/stm/postgres_stm.py`	Durable STM message table, recent/window/flush/stats.	Implemented	Integration tests cover STM->MTM flush.
PostgresMTM	`continuum/stores/postgres/mtm.py`	MTM blocks in `memory_nodes`, recent summaries, unprocessed scan, processed markers, dedupe.	Implemented	Stores MTM as layer='MTM'.
Legacy MTMRepository	`memory/mtm/repository/mtmRepository.py`	Legacy psycopg2 MTM sessions/memories repository.	Partial	Excluded from wheel via `pyproject.toml`.
Legacy MTMRetriever	`memory/mtm/repository/mtmRetriever.py`	Vector retrieval over legacy MTM memories with importance/recency/session boost.	Partial	Functional legacy path, not packaged as modern runtime.
PostgresLTM	`continuum/stores/postgres/ltm.py`	LTM upsert/update/invalidate, hybrid search, graph neighbors.	Partial	Store exists; full source-traced supersession engine is not complete.
LTM schema	`migrations/001_ltm_schema.sql`, `004_policy_engine.sql`	memory_nodes, memory_edges, promotions, episodes, policy columns, superseded_by.	Partial	Schema supports raw episodes and supersession; runtime wiring is limited.
PostgresRetriever	`continuum/stores/postgres/retrieval.py`	Dense, lexical, hybrid search helpers and RRF.	Implemented	Separate utility from high-level Retriever.
Retriever	`continuum/retrieval/retriever.py`	LTM hybrid search, graph expansion, scoring, rerank, STM/MTM assembly.	Implemented	Does not embed queries itself; callers must provide embeddings for dense search.
Scorer	`continuum/scoring/scorer.py`	Relevance, importance, recency, confidence scoring with layer boosts.	Implemented	Default weights: rel 0.45, imp 0.25, rec 0.20, conf 0.10.
Reranker	`continuum/retrieval/reranker.py`	Optional BGE cross-encoder reranking.	Implemented	Lazy loaded and failure-tolerant.
EmbeddingService	`continuum/embeddings/service.py`	Async batched embedding with LRU/Redis cache, FP16 halfvec, CPU fallback.	Implemented	Default code config uses BAAI/bge-m3 1024-dim.

Component	File/Path	Responsibility	Status	Notes
ContextAugmenter	`memory/mtm/contextAugmenter.py`	Legacy prompt assembly from MTM + STM under token budget.	Partial	Legacy only; modern Retriever returns ContextBundle.
OptimizerChain	`continuum/optimizer/`	STM trim, MTM summarize, semantic dedupe, LLMLingua compression, score-aware prune.	Implemented	Optional and not wired by default.
EntityExtractor	`continuum/extraction/entity_extractor.py`	GLiNER NER and relation heuristics.	Implemented	LLM enhancement exists in `llm_extractor.py`.
FactExtractor	`continuum/extraction/fact_extractor.py`	LLM atomic fact extraction from SummaryBlock.	Implemented	Used by promotion pipeline.
MemoryCandidateExtractor	`continuum/extraction/candidate_extractor.py`	Regex/heuristic policy candidates: facts, preferences, tasks, decisions, corrections, procedures, secrets.	Implemented	Not a full semantic preference or temporal engine.
Promoter	`continuum/promotion/promoter.py`	MTM->LTM extraction, neighbor lookup, Mem0-style decisions, write ADD/UPDATE/DELETE/NOOP.	Partial	Promotion exists; knowledge-update semantics remain limited.
Mem0Promoter	`continuum/promotion/mem0_promoter.py`	ADD/UPDATE/DELETE/NOOP decision with cosine short-circuits and LLM tool schema.	Implemented	Audit sink support; degrades to NOOP on LLM failure.
TriggerManager	`continuum/promotion/triggers.py`	Promotion triggers on new entity, block accumulation, periodic scheduling.	Implemented	Optional; tests cover trigger firing.
IdleStmFlush	`continuum/promotion/idle_flush.py`	Flushes partial STM to MTM after idle gap.	Implemented	Cross-session integration test proves plumbing.
Policy Engine	`continuum/policies/`	Candidate classification, policy plans, privacy/update/retrieval/retention decisions, traces, storage routing.	Partial	Default registry function says 8 policies but returns 9; projections for graph/code/raw evidence are mostly tags/no-ops.
Raw memory episodes	`migrations/001_ltm_schema.sql`	`memory_episodes` append-only table for raw evidence.	Partial	Schema present; no dedicated fully wired episode store found.
LLM Wiki	`evals/longmemeval/wiki.py`	Eval-time per-session summary/fact wiki, cached and embedded.	Partial	Eval-only, not core source of truth.
Content Wiki	`evals/longmemeval/content_wiki.py`	Hand-tuned topic wiki pages for benchmark evidence.	Partial	Overfit evaluation experiment.
LongMemEval harness	`evals/longmemeval/`	Adapters, telemetry, trace JSONL, answer validators, candidate extractors, scorers.	Implemented	Benchmark harness bypasses real MTM/LTM promotion in several runs.
Memory-operation benchmarks	`bench/`, `bench/results/`	Ingest, retrieval, supersession simulation, bi-temporal simulation.	Implemented	Benchmarks include simulated detection paths; interpret carefully.
AgentSTM / AgentMTM	`agent/AgentSTM.py`, `agent/AgentMTM.py`, `main.py`	Legacy CLI agents over STM/MTM with optional web search.	Partial	Legacy and not shipped in wheel.
API server	`api/main.py`	FastAPI surface placeholder.	Planned	File is empty in current repo.
MCP web search	`continuum_mcp/web_search_server.py`	FastMCP tools for web search/fetch.	Partial	Auxiliary grounding tool; not packaged in modern wheel.
Continuum-Code	`continuum/core/config.py`, `CodeMemoryPolicy`, docs	Future code knowledge graph/indexing configuration and policy.	Planned	Docs explicitly say code subsystem is not yet implemented.

CHAPTER 3 - High-Level Architecture Diagram

The diagram below shows the intended architecture using only components that are implemented, partially implemented, planned, or recommended by the repo evidence. Not all boxes run in the default quickstart; the default session wires STM and a stub responder only.



TRADE-OFF

The repo favors modular interfaces over a monolithic "memory OS." That makes components easy to test and replace, but it requires explicit wiring. In an interview, say this was intentional for framework adoption, not an accident.

CHAPTER 4 - Memory Lifecycle Deep Dive

STM

Implemented

Raw recent turns, session-local working context, eviction/compression hooks, thread-safe wrappers, Postgres persistence option.

MTM

Implemented

Intermediate blocks/summaries that can be scanned for promotion and included in context under a budget.

LTM

Partial

Durable facts and entities with confidence, importance, validity windows, invalidation, hybrid search, and graph edges. Full update semantics still need work.

STM - Short-Term Memory

Purpose. STM holds immediate conversational state. It should be cheap to write, fast to read, and safe to discard or flush when it becomes old or too large.

Actual implementation. The legacy `ConversationSTM` has bounded message storage, token counting, importance, eviction, compression hooks, rollback, and stats. The modern `InMemorySTM` wraps it behind async STMProtocol. `PostgresSTM` persists raw messages in a database table. Thread-safe wrappers exist in both legacy and modern code.

Data model. Modern STM exchanges `MemoryItem` objects with `content`, `session_id`, `agent_id`, `user_id`, metadata role, and tier. Legacy STM uses `Message`, `STMConfig`, `STMCallbacks`, and `Importance`.

Read/write behavior. `ContinuumSession.process_turn` appends the user message, retrieves context if a retriever exists, calls the responder, then appends the assistant reply. `get_recent` and `window` serve recency reads. `flush_to` transfers STM content to MTM.

Retrieval behavior. High-level `Retriever` includes recent STM turns at the end of the assembled context. It uses `config.stm_turns` and the query session id.

Lifecycle. STM starts as raw turns, may be compressed or evicted in legacy code, may be flushed to MTM on idle or explicit call, and may be cleared per session.

Failure modes. Unbounded STM grows context cost. Premature flush can lose in-progress conversational continuity. Session id mistakes cause cross-session leakage. Compression can lose important facts.

INTERVIEW EXPLANATION

STM is the working set. It lets the agent remain conversational without polluting durable memory. I can delete or flush STM without corrupting long-term truth because it is intentionally short-lived.

MTM - Medium-Term Memory

Purpose. MTM is probation memory. It bridges raw turns and permanent facts by storing summaries or blocks that are useful across a session boundary but not yet trusted as durable truth.

Actual implementation. `PostgresMTM` stores MTM records in `memory_nodes` with `layer='MTM'`. It can add summaries, fetch recent summaries within a token budget, scan unprocessed blocks, mark processed blocks through `memory_promotions`, and optionally dedupe near-duplicates on write. Legacy `MTMRepository` and `MTMRetriever` exist but are excluded from the wheel.

Data model. MTM uses `SummaryBlock` and `MemoryItem`. Metadata includes session id, agent id, role, tokens, source ids, and tags. Legacy MTM has separate `mtm_sessions` and `mtm_memories`.

Read/write behavior. STM can flush into MTM. The promoter scans unprocessed MTM blocks, extracts entities/facts/candidates, asks the decider for ADD/UPDATE/DELETE/NOOP, then marks MTM blocks processed.

Retrieval behavior. High-level Retriever uses `mtm.recent(token_budget, session_id=...)`, converts blocks to memory items, and inserts them between LTM and STM in the final ContextBundle.

Lifecycle. MTM accumulates session summaries, dedupes, feeds promotion, and provides cross-session context before facts are canonicalized.

Failure modes. MTM can become a junk drawer if promotion does not run. Summary quality can be weak. Over-dedupe can collapse distinct events. Under-dedupe causes duplicate pollution.

CHOSEN DESIGN

MTM exists to avoid direct permanent writes. It lets Continuum defer the decision of what should become LTM until extraction, policy, and contradiction checks have a chance to run.

LTM - Long-Term Memory

Current repo status. LTM is partially implemented. The store, schema, hybrid retrieval, invalidation, validity filters, and graph traversal exist. A full source-traced promotion policy that reliably supersedes old facts and writes `superseded_by` is not fully wired.

Actual implementation. `PostgresLTM` writes to `memory_nodes`. `upsert` inserts or updates a row by id. `update` mutates allowlisted columns. `invalidate` sets `invalidated_at`; it does not delete. `search_hybrid` performs dense + sparse RRF and filters invalidated rows and optional `as_of` validity windows. `neighbors` traverses `memory_edges`.

Data model. LTM rows include text, layer, kind, confidence, importance, embedding, source ids, tags, valid_from, valid_to, invalidated_at, and policy columns from migration 004. `memory_edges` supports graph relationships. `memory_episodes` supports raw evidence but is not fully wired as a first-class runtime store.

Recommended LTM design. Durable LTM should store facts, preferences, entity-linked claims, source evidence references, confidence, validity period, last verified timestamp, contradiction state, and promotion provenance from MTM. This should be implemented as append-friendly versioned records, not silent overwrites.

Failure modes. If LTM accepts everything, it becomes noisy and stale. If it overwrites silently, auditability is lost. If supersession is partial, old facts can outrank current facts. If promotion confidence is too low, hallucinated or temporary statements become permanent.

> INTERVIEW EXPLANATION
>
> LTM is not just a vector table. It is where memory becomes durable, typed, traceable, and governed. The current code has the storage and retrieval substrate; the remaining hard part is higher-quality promotion and update semantics.

CHAPTER 5 - Retrieval Architecture

The actual retrieval path is layered. `Retriever.retrieve` can run optional HyDE, LTM hybrid search, graph expansion, composite scoring, policy filtering/boosting, optional reranking, MTM recent summary retrieval, STM recent retrieval, and ContextBundle assembly. `PostgresLTM.search_hybrid` uses dense halfvec search when `Query.embedding` exists and lexical pg_trgm search when text exists, then fuses ranks with RRF.

> **CURRENT REPO STATUS**
>
> The Retriever does not call `EmbeddingService` internally. Dense LTM retrieval requires the caller to supply `Query.embedding`. Without an embedding, PostgresLTM can still use lexical search. This is an important wiring gap to mention honestly.

Why Pure Cosine Can Have High Recall But Poor Accuracy

Cosine similarity answers "which text is semantically near this query?" It does not know which memory is current, which entity is the right one, whether the query asks for a count, whether an older statement was superseded, or whether a preference is different from a fact. The LongMemEval findings show the same pattern: retrieval can surface the right session, but the model still gives the wrong answer because ranking, exact extraction, temporal reasoning, numeric handling, and answer shaping remain weak.

Recommended Scoring Model

```
final_score =
semantic_similarity
+ entity_match_score
+ temporal_relevance_score
+ memory_type_match_score
+ user_or_project_scope_score
+ specificity_score
+ freshness_score
- contradiction_penalty
- stale_memory_penalty
```

Score Term	Meaning	Why It Matters
semantic_similarity	Embedding or lexical relevance to query.	High recall baseline; not enough by itself.
entity_match_score	Explicit match on people, projects, files, products, locations, or code symbols.	Prevents "similar but wrong entity" retrieval.
temporal_relevance_score	Match to as-of, latest, before/after, currently, or historical intent.	Separates old facts from current facts.
memory_type_match_score	Preference queries prefer preference memories; task queries prefer task memories.	Avoids treating every memory as the same kind of chunk.
user_or_project_scope_score	User, tenant, project, repo, agent, or session scoping.	Prevents cross-user and cross-project contamination.
specificity_score	Exact, value-bearing fact beats broad summary.	Improves answer precision for numbers, dates, names.
freshness_score	Boost recent facts where currentness matters.	Useful for changing preferences, tasks, status, and temporal questions.
contradiction_penalty	Penalize facts known to conflict with stronger or newer evidence.	Protects against duplicate contradictory memories.
stale_memory_penalty	Penalize expired, superseded, invalidated, or old-current facts.	Prevents stale memory from shaping current answers.

Option	Strength	Weakness	Why Chosen / Not Chosen for Continuum
Pure cosine similarity	Simple, high recall for semantically related text.	Weak on exactness, temporal state, contradictions, entities.	Not chosen as final architecture. Used as a baseline in eval and benchmark paths.
Pure vector DB retrieval	Scales ANN search; common RAG primitive.	Vector DB alone is not source of truth, policy, lifecycle, or update engine.	Not chosen as source of truth. pgvector is used as one retrieval channel.
Pure keyword / BM25	Excellent for names, IDs, exact terms.	Poor semantic recall and paraphrase handling.	Not chosen alone. Lexical pg_trgm is included in hybrid search.
LLM Wiki-only retrieval	Compressed, readable, fast to scan.	Can go stale, hallucinate compiled claims, and lose source granularity.	Not chosen as sole memory. Eval-only wiki exists as an optional derived layer.
Knowledge graph-first	Powerful for multi-hop relationships and code dependency questions.	High schema/entity-resolution cost; over-engineering risk.	Not chosen for v1 core. Schema edges exist as partial optional layer.
Long-context stuffing	Maximizes recall by including everything.	Expensive, noisy, and findings show 100% recall did not improve accuracy on LongMemEval.	Not chosen as product strategy.

Option	Strength	Weakness	Why Chosen / Not Chosen for Continuum
Agentic iterative retrieval	Can decompose and gather evidence across sessions.	More latency, cost, orchestration complexity, and caller coupling.	Planned or fallback. Eval DecompositionRetriever exists; not default runtime.
Hybrid retrieval	Combines semantic recall, lexical precision, metadata, recency, policy, rerank.	More moving parts and requires calibration.	Chosen. Implemented through PostgresLTM/PostgresRetriever/Retriever.

CHAPTER 6 - Raw Memory Ledger vs Derived Memory

The chosen direction is that the raw memory ledger should be the source of truth. The repo partly supports this through `memory_episodes`, source ids, source block ids, source refs/spans in policies, and promotion audit tables. The full raw-ledger runtime is still partial because there is no dedicated episode store wired through the main session path.

CHOSEN DESIGN

Raw Memory Ledger + derived retrieval artifacts. Keep original evidence so facts, summaries, wiki pages, and embeddings can be regenerated, audited, and corrected.

NOT CHOSEN

Vector DB as source of truth, LLM Wiki as source of truth, summary-only memory, prompt-only memory, or cache-only memory.

Raw memory must be retained because derived memory is lossy. A vector is not evidence. A summary may over-compress. A wiki page may hallucinate or fail to update. A cache may expire. Source traceability matters when an interviewer asks, "Why should I trust the memory?" The answer is: because every durable claim should be linked back to the raw turn, document, tool result, or episode that produced it.

INTERVIEW ANSWER - WHY NOT JUST STORE SUMMARIES?

Summaries are useful, but they are derived artifacts. If we store only summaries, we lose the ability to audit the original user statement, re-extract facts with a better extractor, resolve contradictions, or explain why a memory exists. Continuum should store raw episodes as the ledger, then build summaries, vectors, wiki pages, and graph edges from that ledger. That gives us compression without making compression the source of truth.

CHAPTER 7 - LLM Wiki / Compiled Memory Layer

The repo has an LLM Wiki implementation in the LongMemEval harness, not as the canonical runtime memory layer. `evals/longmemeval/wiki.py` builds per-session wiki entries with a summary and atomic facts, caches them by prompt/content hash, embeds them, and exposes a `WikiRetriever` that returns a ContextBundle. `bootstrap_ollama.py` also contains wiki-memory and content-wiki modes.

This is best classified as Partially implemented: implemented for evaluation experiments, not chosen as the core source of truth.

Wiki Function	Benefit	Risk	Required Guardrail
Compress sessions into readable pages	Faster retrieval and smaller context.	Over-compression and missing facts.	Keep raw evidence and source ids.
Expose atomic facts	Improves exact evidence quality.	Compiled claim may be wrong.	Every claim points to raw turn/session.
Topic pages	Can group scattered facts.	Topic extractor may be overfit.	Use general extractors and validation, not benchmark-specific rules.
Retrieval acceleration	Search fewer, denser artifacts.	Stale pages can beat current raw facts.	Invalidate/rebuild on source changes and use freshness penalties.

FINAL POSITIONING

Continuum should not be wiki-only memory. The defensible architecture is Raw Memory Ledger + Hybrid Retrieval + Optional Compiled Wiki Layer. The wiki is an index and compression artifact, not the database of record.

CHAPTER 8 - Context Augmentation and Optimization

Context enters prompts through two paths. The modern path is `Retriever._assemble`, which orders LTM facts, MTM summaries, and STM recent turns into a `ContextBundle` with prompt-ready messages and token breakdown. The optional `OptimizerChain` can then apply STM trimming, MTM summarization, semantic dedupe, LLMLingua compression, and score-aware pruning. The legacy path is `memory/mtm/contextAugmenter.py`, used by `AgentMTM` to build a prompt with MTM memories and STM messages under a token budget.

Capability	Repo Evidence	Status	Notes
Insert retrieved memory into context	`Retriever._assemble`, `ContextAugmenter.build_context`	Implemented	Modern and legacy paths exist.
Duplicate handling	`SemanticDedupe`, MTM dedupe threshold	Implemented	Optional optimizer and MTM write-time dedupe.
Token budget	`TokenBudget`, optimizer strategies, legacy max_context_tokens	Implemented	Budgeting exists but not all paths enforce hard final cap.
Summarization	`StmTrim`, `MtmSummarize`, legacy STM compression hooks	Implemented	LLM summarizer is injectable/optional; extractive fallback exists.
Fact priority	`Scorer`, policy boosts, score-aware prune	Implemented	Priority is present but scoring needs calibration.
Pluggable optimizer interface	`OptimizerProtocol`, `BaseOptimizer`	Implemented	Good interview point: context quality is modular.

NOT CHOSEN FOR V1

TurboQuant-like low-level inference optimization, custom quantization algorithms, model-level optimization, or GPU/runtime optimization. Continuum's core value is memory quality and context assembly. Model compression is a separate research/product area and should not block the memory layer.

CHAPTER 9 - Knowledge Graph Decision

The repo includes partial graph support: `memory_edges`, `PostgresLTM.neighbors`, graph expansion in `Retriever`, and a `GRAPH_INDEX` projection flag in the policy layer. What is missing is a full graph-first ingestion engine with robust entity resolution, edge extraction, schema governance, and maintenance.

NOT CHOSEN AS DEFAULT V1 CORE

A graph-first memory architecture was not selected as the main v1 path. It creates high schema complexity, entity-resolution complexity, edge maintenance cost, harder ingestion, harder debugging, slower iteration, and over-engineering risk before retrieval quality is stable.

CHOSEN

Metadata + embeddings + source traceability first, with optional graph expansion later. This lets Continuum ship a useful memory layer while preserving a path to graph reasoning where it matters.

Graph is valuable for code repository understanding, dependency maps, object relationships, entity relationship reasoning, multi-hop queries, and future Continuum-Code. The current repo has config and policy hooks for code memory, but docs explicitly say the code indexing subsystem is not implemented.

INTERVIEW ANSWER - WHY NOT START WITH A KNOWLEDGE GRAPH?

I would not start graph-first because the hard early problem is memory lifecycle and evidence quality, not graph traversal. A graph makes sense after entity extraction, source traceability, update semantics, and retrieval quality are stable. Otherwise you spend most of the project debugging entities and edges instead of building a reliable memory product. Continuum keeps graph support optional so we can add it where it has clear value, especially for code and multi-hop entity queries.

CHAPTER 10 - Temporal, Preference, and Knowledge Update Engines

The repository has building blocks, not full engines. Temporal fields exist in schema and query filters. Preferences are recognized by candidate extraction and policies. Corrections and supersession vocabulary exist in policies and migrations. Eval helpers classify temporal, preference, and knowledge-update questions. But the dedicated engines are marked missing or partial in `docs/continuum_eval_fix_plan.md`.

Engine	Repo Evidence	Status	What Is Missing
Temporal engine	`valid_from` / `valid_to`, `Query.as_of`, bi-temporal benchmark simulation, eval question classifier	Planned	Dedicated dated-event extraction, duration arithmetic, ordering, latest/current reasoning.
Preference engine	`UserPreferencePolicy`, candidate extractors, eval `extract_preferences`	Partial	Preference reasoning over multiple statements, strength, polarity, and updates.
Knowledge-update engine	`CorrectionPolicy`, `superseded_by` schema, eval `extract_change_facts`	Partial	Old/new/current resolver, source confidence, overwrite/version policy, contradiction state.

A temporal engine handles words like latest, current, before, after, during, used to, previously, now, and changed from X to Y. A preference engine separates user preferences from facts, events, tasks, project knowledge, and temporary statements. A knowledge-update engine handles contradictions, replacement facts, stale facts, source confidence, overwrite policy, and versioning.

> **WHY VECTOR SEARCH ALONE CANNOT SOLVE THIS**
>
> Vector search can find related memories, but it cannot reliably decide which fact is current, whether an older statement is superseded, whether a statement is a stable preference or a temporary mood, or whether a contradiction should overwrite, version, ask the user, or keep both.

CHAPTER 11 - Evaluation and Telemetry

The repo contains two evaluation tracks. First, LongMemEval experiments under `evals/longmemeval/` measure answer accuracy, recall, latency, token/cost estimates, failure categories, question types, trace rows, candidate extraction, answer cleanup, validation, decomposition, wiki experiments, and hybrid scoring. Second, memory-operation benchmarks under `bench/` measure ingest overhead, synthetic retrieval quality, supersession simulation, and bi-temporal simulation.

Artifact	What It Measures	Status	Notes
`evals/longmemeval/baseline.py`	Accuracy, recall, latency p50/p95, cost, failure categories, row results.	Implemented	Persists JSON and failure CSV.
`evals/longmemeval/telemetry.py`	Per-row LLM calls, tokens, cost, retrieval counts, validator state.	Implemented	ContextVar per-row counter.
`evals/longmemeval/trace.py`	Append-only JSONL per-row traces with stable schema.	Implemented	Supports debugging wrong answers.
`answer_post.py`	Final answer cleanup, shape validation, repair prompts, don't-IDK guard.	Implemented	Evaluation answer-quality layer.
`candidates.py`	Regex candidates for counts, dates, durations, locations, preferences, old/new facts.	Implemented	Eval-specific but useful diagnostic design.
`hybrid_scorer.py`	Deterministic exact/normalized/numeric/unit/rule scorer plus optional LLM judge.	Implemented	Addresses substring scorer undercount.
`bench/results/*latest.json`	Latest memory-operation metrics.	Implemented	Includes ingest, retrieval, supersession, bi-temporal results.

Repo-contained benchmark context: `findings/longmemeval_2026-05.md` reports six full LongMemEval-S runs at 500 questions each. Accuracy is bounded at roughly 26.2% to 34.4% for substring scoring despite varied models and retrieval modes. The report states the failure mix is about 90% `wrong_retrieval`: the right session was retrieved but the model still answered incorrectly. `results/optimizer_iteration_3_retrieval_tuning.json` also records an earlier baseline row for `llama-3.2:3b` with accuracy 29.4% and recall 87.5%.

> **INTERPRETATION**
>
> High recall with low accuracy means retrieval is finding related memories, but ranking, exactness, answer shaping, temporal handling, numeric handling, contradiction handling, or reasoning is still weak. This is a strong interview point because it shows evaluation changed the architecture priorities.

Recommended Improvement Phases

Phase	Status / Action	Why It Matters
A4 - telemetry	Implemented in eval harness; keep extending.	Debugging requires per-row evidence, not aggregate accuracy alone.
A0 - fixed 50-row diagnostic sample	Implemented by `scripts/build_diagnostic_sample.py`.	Cheap regression loop before full 500-row runs.
A1 - don't-IDK guard	Implemented in answer post-processing.	Prevents refusal when candidates exist.
A2 - numeric/count aggregator patch	Implemented in eval candidates/aggregation path.	Count and numeric answers are frequent exactness failures.
A3 - output scaffold cleanup	Implemented in `clean_final_answer`.	Reduces benchmark false negatives from debug text.
A-lite validator	Implemented as answer-shape checks.	Stops location/date/count answers from having the wrong shape.
Gate A	If Phase A does not reach 50-55%, inspect failure mix before pivoting.	Avoid blind feature work.
B1 - temporal engine	Missing / deferred	Latest/current/as-of failures need explicit logic.
B3 - knowledge-update engine	Partial	Old/new/current resolution is central to memory correctness.
B2 - preference engine	Partial	Preferences must not be treated as ordinary facts.
Gate B	If Phase B adds less than +10 percentage points, pivot to agentic iterative retrieval / evidence expansion.	Prevents over-investing in a weak lever.
C - full validator + release scoring	Recommended	Turns eval from experiments into release gates.

CHAPTER 12 - Chosen vs Not Chosen Architecture Decision Records

ADR 1 - STM -> MTM -> LTM lifecycle

Decision	Use staged memory lifecycle.
Context	Agents produce raw, noisy, conflicting turns.
Options considered	Staged lifecycle, flat table, session-only, direct permanent writes.
Chosen	STM -> MTM -> LTM.
Not chosen	Flat memory table, session-only memory, direct permanent writes.
Why chosen	Controls promotion quality and memory pollution.
Trade-off	More moving parts and background processing.
Status	Partial end-to-end, with tiers implemented.
Future	Stronger promotion policy and LTM rollout.
Interview answer	Lifecycle is the main difference from simple RAG.

ADR 2 - Raw Memory Ledger

Decision	Raw evidence should be source of truth.
Context	Derived summaries and vectors are lossy.
Options considered	Raw ledger, vector DB, wiki, summary-only.
Chosen	Raw ledger with derived artifacts.
Not chosen	Vector DB/wiki/summary as source of truth.
Why chosen	Auditability and reprocessing.
Trade-off	More storage and provenance tracking.
Status	Partial; schema exists.
Future	Dedicated episode store and source links.
Interview answer	Evidence survives extractor upgrades and disputes.

ADR 3 - Hybrid Retrieval

Decision	Combine semantic, lexical, metadata, scoring, temporal, entity, rerank.
Context	Pure cosine fails exact/current questions.
Options	Pure cosine, BM25, vector-only, hybrid.
Chosen	Hybrid retrieval.
Not chosen	Pure cosine, pure BM25, vector-only.
Why chosen	Balances recall and precision.
Trade-off	Calibration complexity.
Status	Implemented core hybrid; expanded scoring recommended.
Future	Add temporal/entity penalties and boosts.
Interview answer	Memory retrieval needs state, not just similarity.

ADR 4 - LLM Wiki as Derived Layer

Decision	Use compiled wiki as optional derived layer.
Context	Long memory needs compression.
Options	Wiki-only, no wiki, derived wiki.
Chosen	Optional compiled memory layer.
Not chosen	Wiki-only architecture.
Why chosen	Compression without losing audit trail.
Trade-off	Staleness and rebuild complexity.
Status	Partial eval-only.
Future	Core wiki backed by raw evidence.
Interview answer	Wiki is an index, not truth.

ADR 5 - Knowledge Graph

Decision	Keep graph optional/future.
Context	Graph helps multi-hop but raises complexity.
Options	Graph-first, optional graph, no graph.
Chosen	Optional/future graph layer.
Not chosen	Graph-first core.
Why chosen	Avoid over-engineering before extraction/retrieval mature.
Trade-off	Some multi-hop queries remain weak.
Status	Partial schema/traversal.
Future	Continuum-Code and entity graph.
Interview answer	Graph after evidence quality.

ADR 6 - Context Optimizer

Decision	Optimize context assembly and token budgeting.
Context	Memory quality enters the model through the prompt.
Options	Prompt optimizer, model quantization, GPU runtime work.
Chosen	Pluggable context assembly.
Not chosen	Low-level model optimization in v1.
Why chosen	Directly improves answer grounding and cost.
Trade-off	May lose facts if pruning is too aggressive.
Status	Implemented optional chain.
Future	Stronger budget-aware factual selector.
Interview answer	The product value is context quality.

ADR 7 - Validation

Decision	Validate answer shape and release quality.
Context	Raw LLM output can be wrong-shaped.
Options	Trust LLM, progressive validator.
Chosen	Validator and release scoring.
Not chosen	Trusting raw LLM output only.
Why chosen	Improves exactness and debuggability.
Trade-off	Rules can be brittle.
Status	Partial eval implemented; runtime recommended.
Future	Core validator API.
Interview answer	Memory systems need answer contracts.

ADR 8 - Evaluation

Decision	Benchmark + telemetry + failure classification.
Context	Memory quality is not visible from demos.
Options	Manual subjective testing, instrumented eval.
Chosen	Evaluation-driven iteration.
Not chosen	Manual testing only.
Why chosen	Surfaces recall/accuracy mismatch.
Trade-off	Benchmarks can be noisy or misaligned.
Status	Implemented
Future	Release gates in CI.
Interview answer	Failure mix tells us where to invest.

ADR 9 - Temporal Handling

Decision	Use explicit temporal engine.
Context	Latest/current/as-of are common memory queries.
Options	Vector inference, explicit temporal logic.
Chosen	Explicit temporal engine.
Not chosen	Relying on vector search to infer time.
Why chosen	Temporal semantics are stateful.
Trade-off	Requires date extraction and validity logic.
Status	Planned; schema support partial.
Future	B1 temporal engine.
Interview answer	Similarity does not know "current."

ADR 10 - Preference Handling

Decision	Separate preference-aware handling.
Context	Preferences update differently than facts.
Options	Single memory type, preference-aware policy.
Chosen	Preference-aware memory handling.
Not chosen	Treating all memory as same type.
Why chosen	Preferences need polarity, strength, recency.
Trade-off	More extraction complexity.
Status	Partial
Future	B2 preference engine.
Interview answer	A preference is not just a fact row.

ADR 11 - Contradiction Handling

Decision	Source-traced update engine.
Context	Users correct themselves and facts change.
Options	Silent overwrite, duplicates, supersession.
Chosen	Source-traced update/supersession.
Not chosen	Silent overwrite or duplicate contradictions.
Why chosen	Preserves audit and currentness.
Trade-off	Detection is hard.
Status	Partial
Future	B3 knowledge-update engine.
Interview answer	Overwrite loses history; duplicates confuse retrieval.

ADR 12 - Agentic Retrieval

Decision	Use as fallback/future strategy.
Context	Multi-hop questions need multiple evidence reads.
Options	Default agentic loop, single-shot, fallback.
Chosen	Fallback/future strategy.
Not chosen	Default v1 retrieval path.
Why chosen	Keeps base latency and API simple.
Trade-off	One-shot accuracy ceiling remains.
Status	Partial eval DecompositionRetriever.
Future	Evidence expansion and iterative planner.
Interview answer	Separate memory layer from reasoner.

ADR 13 - Storage

Decision	Postgres/pgvector current storage with adapters.
Context	Need structured metadata plus vectors.
Options	One vendor DB, graph DB, vector DB, Postgres adapter.
Chosen	PostgreSQL/pgvector or current repo storage with interface-first adapters.
Not chosen	Hard dependency on one vector DB/graph DB vendor.
Why chosen	SQL metadata, transactions, JSONB, vector, trigram in one place.
Trade-off	May need split storage at very large scale.
Status	Implemented Postgres stores.
Future	Adapter split for external vector stores.
Interview answer	Postgres gives correctness before specialization.

ADR 14 - Interface-First Design

Decision	Modular protocols and injected components.
Context	Users will bring different LLMs/stores/retrievers.
Options	Tightly coupled implementation, modular interfaces.
Chosen	Interface-first design.
Not chosen	Tightly coupled implementation.
Why chosen	Testability and replaceability.
Trade-off	More wiring required.
Status	Implemented
Future	Factory presets for common stacks.
Interview answer	Frameworks should compose, not own the app.

ADR 15 - LTM Rollout

Decision	Incremental promotion pipeline.
Context	Big-bang memory OS is risky.
Options	Full memory OS, incremental pipeline.
Chosen	Incremental promotion pipeline.
Not chosen	Big-bang memory OS implementation.
Why chosen	Lets evaluation drive each layer.
Trade-off	Some planned features are incomplete today.
Status	Partial
Future	Promotion gates, confidence scoring, source ledger.
Interview answer	Ship reliable layers before broad claims.

CHAPTER 13 - Interview Defense Questions

1. Is Continuum just RAG?

No. RAG is usually retrieval over documents or chunks followed by prompt insertion. Continuum is designed around memory lifecycle: STM for recent raw turns, MTM for intermediate summaries, and LTM for durable facts with metadata, confidence, validity, source references, and policy handling. The repo still uses retrieval, embeddings, and context assembly, but those are pieces of a memory system. The main design difference is that memory can be promoted, invalidated, deduped, governed, and evaluated over time instead of being treated as a flat set of chunks.

2. Why not just use vector search?

Vector search is good at semantic recall, but it cannot decide if a fact is current, stale, contradicted, user-specific, project-specific, or a preference rather than a fact. Continuum uses vector search as one channel, not as the architecture. The repo has dense search, lexical search, RRF, composite scoring, policy boosts, and planned temporal/update engines because correctness depends on state and metadata, not similarity alone.

3. Why do you need STM, MTM, and LTM?

They solve different lifecycle problems. STM is the working set for the current conversation. MTM is probation memory: useful summaries or blocks that can survive a session boundary but are not yet permanent truth. LTM is durable, typed, source-backed memory. If you skip MTM and write directly to LTM, temporary statements pollute permanent memory. If you skip LTM, the system never builds reliable cross-session knowledge.

4. Why not store everything permanently?

Because not everything the user says is a durable fact. Some statements are temporary, speculative, wrong, sensitive, or contradicted later. Permanent storage also increases retrieval noise and token cost. Continuum's staged design lets the system keep raw evidence, summarize it, and promote only high-value memory with policy and confidence checks.

5. Why not use a knowledge graph from day one?

A graph is useful after you have reliable entities and facts. Starting graph-first creates a heavy burden: schema design, entity resolution, edge extraction, edge updates, and debugging. The repo already has graph schema and traversal hooks, but the core v1 path is metadata plus embeddings plus source traceability. That is a more pragmatic foundation, and graph can be added for code and relationship-heavy domains.

6. Why not fine-tune the model for memory?

Fine-tuning changes model behavior, but it is not a good way to store user-specific, changing, auditable memory. User memory changes daily and needs deletion, correction, source traceability, access control, and tenant isolation. A memory layer can update instantly and explain where a fact came from. Fine-tuning might help answer style or extraction quality, but it should not be the memory database.

7. How do you handle contradictions?

The intended design is source-traced supersession: a new claim can update, supersede, delete, or no-op against existing memory. The repo has a Mem0-style promoter, correction policies, invalidation, `superseded_by` schema, and benchmarks simulating supersession. The honest current status is partial: the full automated knowledge-update engine that reliably writes contradiction state is not complete. I would describe this as the next core reliability milestone.

8. How do you reduce hallucination?

By reducing unsupported context and improving provenance. Continuum should keep raw evidence, attach sources to derived facts, filter stale or restricted memories, dedupe near-duplicates, and validate answer shape. The eval harness already includes answer cleanup, don't-IDK guard, candidate extraction, and shape validation. The production path should absorb those ideas so the model answers from selected evidence instead of vague related context.

9. How do you control token cost?

The repo has TokenBudget, ContextBundle tier breakdowns, STM/MTM/LTM allocation, semantic dedupe, STM trimming, MTM summarization, LLMLingua compression, and score-aware pruning. Architecturally, the goal is to insert the smallest sufficient evidence set, not the largest possible context. The LongMemEval findings support this: long-context 100% recall did not improve accuracy, so stuffing more tokens is not a good product strategy.

10. How do you measure whether memory works?

I would measure both memory operations and answer outcomes. The repo includes LongMemEval accuracy/recall/failure categories and memory-operation benchmarks for ingest, retrieval recall, supersession simulation, and bi-temporal simulation. The key is not just accuracy; it is failure classification. If recall is high and accuracy is low, the bottleneck is not "retrieve more," it may be ranking, reasoning, temporal handling, or answer validation.

11. Why is recall high but accuracy low?

Because retrieval found related evidence, but the answerer failed to select, compose, or format the exact answer. The LongMemEval report says most incorrect answers are wrong_retrieval: the ground-truth session was retrieved, but the model still answered incorrectly. That points to query-time reasoning, temporal/current-fact handling, numeric aggregation, and validator gaps rather than simple recall.

12. How is this different from Mem0, Zep, LangMem, or normal RAG frameworks?

The differentiator I would defend is not "we also store memories." It is the combination of staged lifecycle, raw evidence as source of truth, hybrid retrieval, source traceability, explicit update/temporal roadmap, and evaluation-driven failure analysis. The repo even uses a Mem0-style promotion decision pattern, so I would not claim every mechanism is unique. I would claim the architecture is built to make memory auditable and lifecycle-aware rather than just embedding notes.

13. What would you build next?

I would build the temporal engine, knowledge-update engine, and preference engine before adding fancy retrieval. The evidence says raw recall is not the only problem. Current facts, old facts, preferences, numeric answers, and contradiction resolution need explicit handling. I would also wire raw memory episodes as a first-class ledger and bring eval validators into the runtime path.

14. What are the hardest engineering problems?

The hardest parts are not vector search. They are deciding what becomes permanent, identifying contradictions, grounding every derived claim in raw evidence, resolving temporal intent, preventing cross-user contamination, and evaluating answer correctness when model output is semantically right but not exact-match. These are state and systems problems as much as ML problems.

15. What are the production risks?

Production risks include stale memory, duplicate memory pollution, hallucinated promoted facts, sensitive data storage, tenant leakage, background promotion lag, poor observability, and retrieval precision regressions. The repo has pieces for policies, background queues, traces, and benchmarks, but production readiness requires stronger monitoring, approval workflows, raw evidence audit, and release gates.

16. How would you scale this?

At small and medium scale, Postgres plus pgvector and pg_trgm is defensible because it stores structured metadata, JSONB policy fields, vector search, lexical search, and transactions together. At larger scale, I would partition by tenant/user, use read replicas, move extraction workers off the hot path, and consider a dedicated vector DB while keeping Postgres as the canonical structured ledger. The interface-first design supports that split.

17. How would you debug wrong memory retrieval?

I would inspect the per-row trace: query, expected source ids, retrieved ids, partial recall, selected evidence, candidate types, answer shape, and failure category. If the right source is missing, improve retrieval. If the right source is present but the answer is wrong, improve ranking, evidence selection, temporal logic, numeric aggregation, or validation. The eval trace schema already supports this style of debugging.

18. How would you support multiple users or tenants?

Every memory item should carry user, tenant, project, session, and agent scope, and retrieval must enforce scope before ranking. The current types include session_id, agent_id, and user_id. I would add hard DB-level tenant filters, row-level security where appropriate, per-tenant indexes or partitions at scale, and tests that prove cross-project and cross-agent isolation.

19. How would you prevent stale memory from being used?

Use validity windows, invalidation, supersession, freshness scoring, and contradiction penalties. Do not rely on recency alone. The repo has valid_from/valid_to, invalidated_at, as_of filters, and superseded_by schema, but the robust current-fact resolver is still planned. I would treat stale-memory prevention as a core release gate.

20. How would you explain this project in two minutes?

Continuum is a memory infrastructure layer for AI agents. It separates working conversation state, intermediate summaries, and durable facts so an agent can remember over time without storing every temporary statement as permanent truth. The runtime has async session orchestration, STM/MTM/LTM stores, PostgreSQL/pgvector persistence, hybrid retrieval, composite scoring, graph hooks, promotion decisions, policy handling, context optimization, and an evaluation harness. The most important design choice is that memory is lifecycle-managed and source-traced, not just vectorized. The current repo proves many of the core pieces, while temporal reasoning, preference handling, knowledge-update resolution, and a fully wired raw ledger remain the next reliability milestones.

CHAPTER 14 - Failure Modes and Mitigations

Failure Mode	Cause	Impact	Detection	Mitigation	Current Status
Wrong memory retrieved	Similarity/ranking miss.	Wrong answer.	Trace retrieved ids vs expected ids.	Hybrid retrieval, rerank, entity/scope scoring.	Implemented baseline hybrid; more scoring recommended.
Similar but incorrect memory	Cosine matches broad topic.	Confident wrong answer.	Manual trace review, entity mismatch.	Entity match score and exact lexical boost.	Recommended
Old memory used as current fact	No temporal/update filter.	Stale answer.	Knowledge-update failures.	Temporal engine, supersession, stale penalty.	Partial
Contradictory memories	Append-only facts without update resolution.	Unstable answers.	Conflict traces, duplicate facts.	Source-traced update engine.	Partial
Over-compressed summary	Lossy summarization.	Missing evidence.	Compare summary to raw turns.	Raw ledger, source links, summary quality checks.	Partial
Wiki page stale	Compiled page not rebuilt.	Old claim beats raw current fact.	Wiki cache/version checks.	Invalidate wiki on raw change; keep raw source.	Eval-only
Too many tokens inserted	Over-retrieval and weak pruning.	Cost and distraction.	Context token telemetry.	Optimizer, score-aware prune, dedupe.	Implemented
Low precision despite high recall	Right evidence present but not selected.	Accuracy plateau.	wrong_retrieval failure type.	Answer validators, candidate extraction, reranking.	Partial
Semantic answer marked wrong	Exact-match benchmark limitations.	Undercounted accuracy.	Hybrid scorer / LLM judge.	Deterministic + LLM scoring ladder.	Eval implemented
Numeric/count answer wrong	Model counts mentions or wrong candidates.	Incorrect exact values.	COUNT question failures.	Numeric aggregator, typed candidates.	Eval implemented
Preference confused with fact	Single memory type.	Bad personalization.	Preference question failures.	Preference engine and policy type boosts.	Partial
Temporary statement promoted	Weak promotion policy.	LTM pollution.	Promotion audit review.	MTM probation, volatility policy, confidence gates.	Partial
Entity collision	Same name across users/projects/entities.	Wrong memory source.	Entity/scope mismatch traces.	Tenant/project/entity IDs and disambiguation.	Recommended
Cross-project contamination	Missing scope filter.	Privacy and correctness issue.	Isolation tests.	Hard user/project filters before ranking.	Partial IDs exist
Missing source traceability	Derived facts without source ids.	No audit path.	Rows without source_ref/source_ids.	Raw ledger and mandatory source references.	Partial
Hallucinated memory	LLM extraction invents facts.	False durable memory.	Promotion audit, low source overlap.	Evidence-bound extraction and confidence thresholds.	Partial
Duplicate memory pollution	No dedupe or weak threshold.	Reinforced wrong answers.	Near-duplicate scans.	SemanticDedupe, MTM dedupe, promotion NOOP.	Implemented
Agent retrieves another agent's memory	Agent scope not enforced.	Privacy/correctness breach.	Cross-agent test.	Agent/user/tenant filters.	IDs present
Poor reranking	Cross-encoder mismatch or disabled reranker.	Good candidates appear too low.	Trace rank positions.	Reranker eval and feature-based scorer.	Optional
Weak benchmark validator	Substring/easy exact scoring.	Misleading accuracy.	Judge disagreement.	Hybrid scorer, answer-shape validators.	Eval implemented

CHAPTER 15 - Production Readiness Roadmap

Now

Item	Why It Matters	Complexity	Interview Value	Expected Impact
Stabilize retrieval	Prevents precision regressions.	Medium	Shows data-driven system tuning.	Higher answer reliability.
Add telemetry	Every failure needs evidence.	Low/Medium	Strong debugging story.	Faster iteration.
Add fixed diagnostic set	Cheap repeatable regression loop.	Low	Shows engineering discipline.	Reduced eval cost.
Improve ranking precision	High recall is not enough.	Medium	Explains recall/accuracy gap.	Fewer wrong_retrieval failures.
Add answer-shape validator	Wrong-shaped answers fail even with evidence.	Medium	Shows practical LLM systems work.	Better exactness.
Fix numeric/count aggregation	Count questions are brittle.	Medium	Good concrete example.	Improved COUNT accuracy.
Clean output scaffold	Debug text should not leak into answers.	Low	Shows benchmark hygiene.	Fewer false negatives.
Improve source traceability	Needed for trust and audit.	Medium	Central AI infra point.	Lower hallucinated-memory risk.

Next

Item	Why It Matters	Complexity	Interview Value	Expected Impact
Temporal engine	Handles latest/current/as-of.	High	Core memory correctness.	Better temporal and update questions.
Knowledge-update engine	Resolves old/new/current contradictions.	High	Hardest product differentiator.	Prevents stale facts.
Preference engine	Separates preferences from facts.	Medium/High	Personalization depth.	Better preference answers.
MTM to LTM promotion policy	Controls durable memory quality.	High	Lifecycle defense.	Less LTM pollution.
LTM schema refinement	Durable facts need source, confidence, validity, contradiction state.	Medium	System design rigor.	Cleaner long-term memory.
Contradiction detection	Corrections happen constantly.	High	Shows non-trivial AI infra.	Current-fact reliability.
Memory confidence scoring	Not all memories are equally trustworthy.	Medium	Explains precision choices.	Better ranking and promotion.

Later

Item	Why It Matters	Complexity	Interview Value	Expected Impact
Agentic retrieval fallback	Multi-hop questions need iterative evidence gathering.	High	Shows architecture boundary between memory and reasoner.	Higher multi-session accuracy.
Continuum-Code knowledge graph	Code dependencies are graph-shaped.	High	AI infra / code intelligence story.	Repo understanding and code Q&A.
Advanced optimizer plugins	Different domains need different compression.	Medium	Pluggability story.	Lower token cost.
UI/SDK	Adoption and debugging.	Medium	Productization.	Easier integration.
Multi-tenant isolation	Production privacy requirement.	High	System design readiness.	Safe SaaS deployment.
Compliance/audit layer	Memory stores user data.	High	Enterprise readiness.	Trust and governance.
Production observability	Need live health and failure metrics.	Medium	Operational maturity.	Faster incident response.
Cost dashboard	LLM extraction and context cost can grow.	Medium	Business-aware engineering.	Cost control.

CHAPTER 16 - Final One-Page Interview Summary

What Continuum Is

Continuum is a memory infrastructure framework for AI agents. It is designed around memory lifecycle: STM for raw recent conversation, MTM for intermediate summaries/blocks, and LTM for durable, typed, source-backed facts.

Why the Architecture Is Defensible

The repo implements async session orchestration, STM/MTM/LTM protocols and stores, Postgres/pgvector/pg_trgm hybrid retrieval, composite scoring, optional reranking, context optimization, promotion decisions, policy handling, evaluation traces, and benchmarks. It also keeps components injected and replaceable.

What Was Deliberately Not Chosen

Continuum does not use vector DB as the source of truth, wiki-only memory, summary-only memory, graph-first memory, long-context stuffing, direct permanent writes for every turn, or low-level model compression as the v1 product center.

Why It Is Better Than Simple RAG

Simple RAG retrieves chunks. Continuum manages memory state: what is short-lived, what is probationary, what is durable, what is current, what is source-backed, and what should be excluded or updated.

What Is Implemented Now

Implemented: session orchestration, STM variants, PostgresSTM/MTM/LTM storage, hybrid retrieval, scoring, optional reranking, embedding service, optimizer chain, promotion pipeline, policy engine pieces, eval telemetry, LongMemEval harness, and memory-operation benchmarks. Partial: raw ledger wiring, supersession/update automation, graph construction, preference/temporal engines, LLM Wiki in core runtime. Planned: Continuum-Code and production API/server surface.

Biggest Trade-offs

The modular architecture is testable and extensible, but not turnkey unless components are wired. Hybrid retrieval is more accurate than pure cosine but harder to calibrate. Keeping raw evidence improves trust but adds storage and provenance complexity. Deferring graph-first design avoids over-engineering but delays some multi-hop capabilities.

How to Pitch It

"Continuum is not just RAG. It is a lifecycle-managed memory layer for agents. The architecture separates raw evidence, short-term context, medium-term summaries, and long-term facts, then retrieves with hybrid search and assembles context under a budget. The repo is honest about what is done and what is next: the foundation exists, and the next reliability frontier is temporal reasoning, knowledge updates, preferences, and stricter validation."

Primary files inspected include README.md, docs, migrations, continuum core/runtime packages, legacy memory/agent packages, eval harness, benchmarks, tests, configs, and generated findings/results. See the companion README for regeneration instructions and assumptions.